

Contents

1. Overview.....	1
2. Worker pool block Diagram.....	2
3. Flow chart.....	Error! Bookmark not defined.
4. User Exposed APIs (inc/workerpool.h).....	3
5. Test suite validation	7
6. Known Limitations	7

1. Overview

The Worker Pool Library is a modular, plugin-style C library that provides a flexible and efficient way to manage concurrent job execution using threads. It is designed to be lightweight, portable, and easy to integrate into any project.

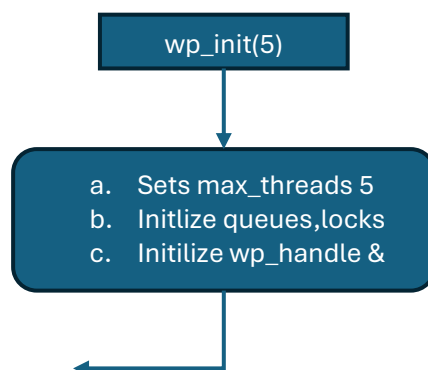
Key Features

- **Configurable Thread Pool Size**
 - The user can decide the maximum number of threads in the pool during the `wp_init()` call.
 - This allows fine-tuning based on workload and system resources.
 - Example: `wp_init(8)` creates a pool with up to 8 worker threads.
- **Multiple Pools in a Single Process**
 - The design supports creating multiple independent worker pools within the same process instance.
 - Each pool maintains its own thread list, job queues, and synchronization primitives.
 - This enables modular usage — different subsystems can run their own pools without interference.
- **Concurrency Protection**
 - All job submissions, completions, and thread state transitions are concurrency-safe.

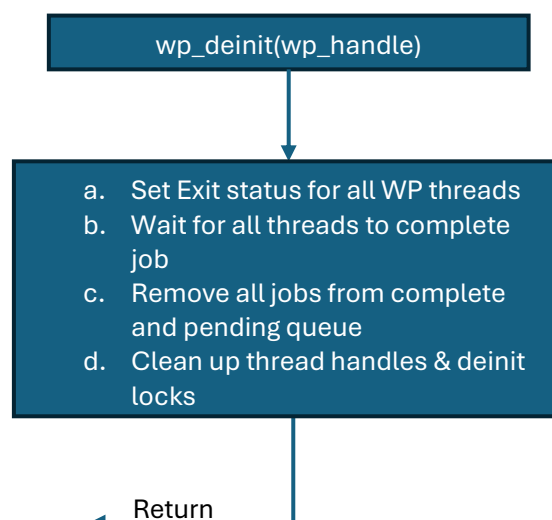
- Internal synchronization (mutexes, semaphores) ensures that jobs are executed correctly even when multiple threads post or collect jobs simultaneously.
- Exception handling is built in: if a thread crashes (e.g., segmentation fault), the pool records the failure, updates the job completion list, and reduces the active thread count safely.
- **Plugin-Style Integration**
 - The library is designed as a standalone module.
 - Developers can include `inc/workerpool.h` and link against the implementation to immediately use the pool in their applications.
 - No external dependencies beyond standard POSIX threads.

2. Worker pool diagrams & flow chart

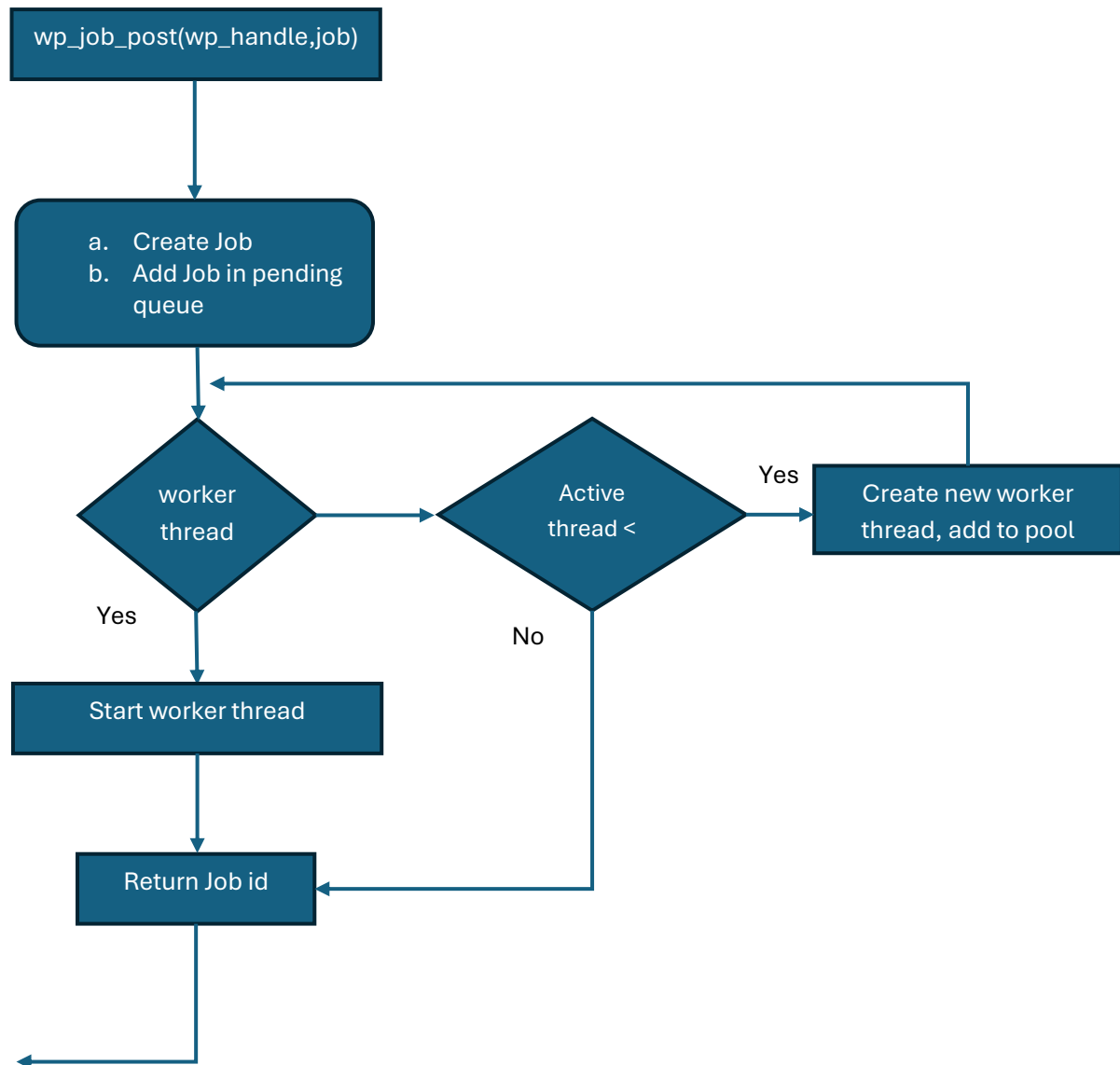
a. Worker pool Initialization Flow



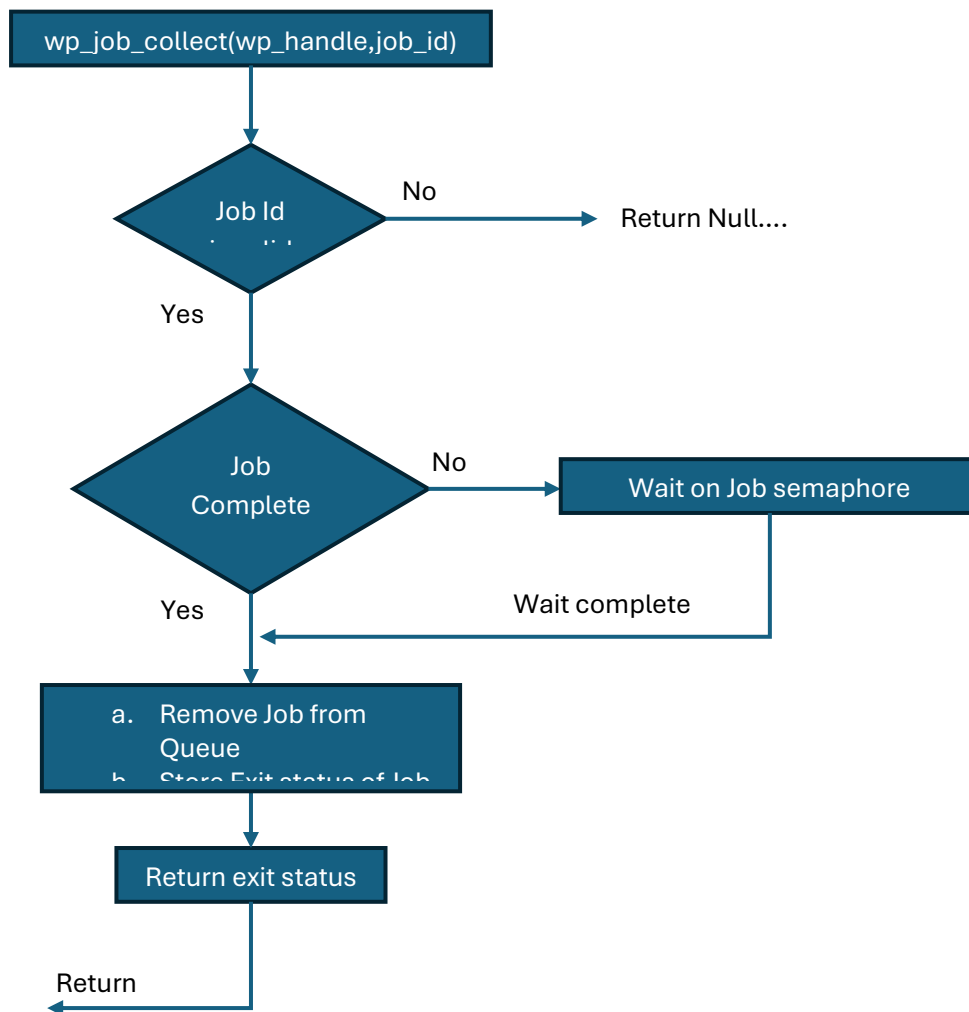
b. worker pool deinit



c. Worker pool Job post API

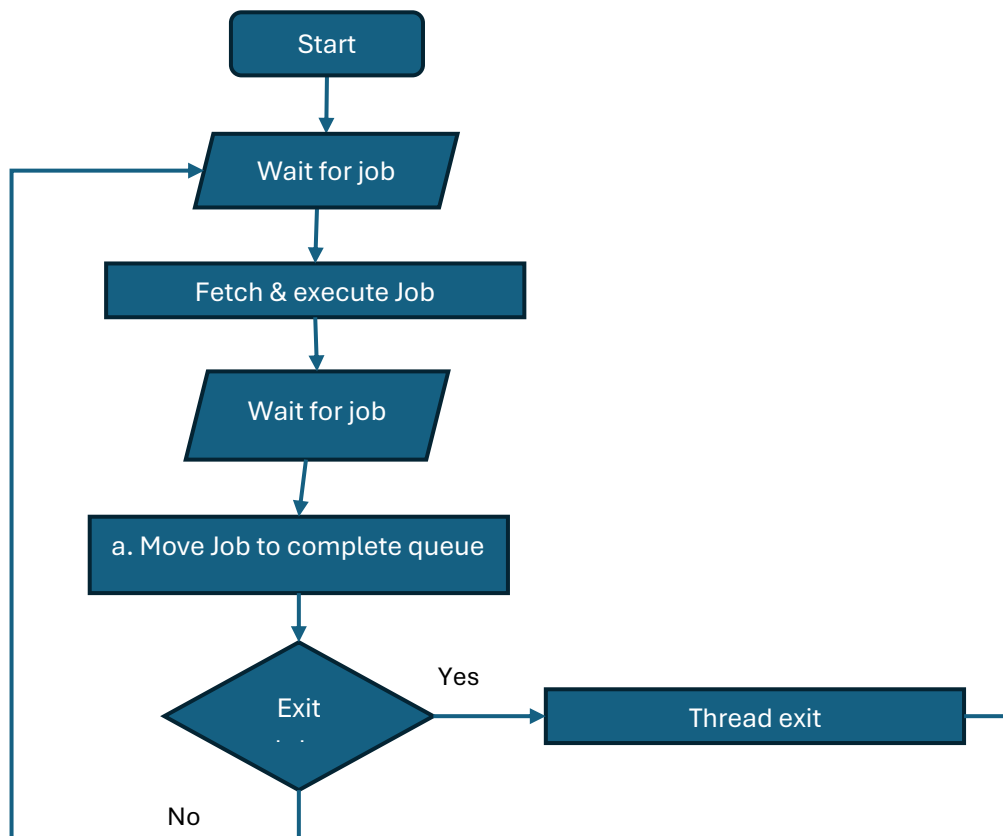


d. worker pool job collect



e. worker pool thread routine

Worker Pool Thread Routine



3. User Exposed APIs (inc/workerpool.h)

int wp_init(int max_threads);

- Args: max_threads → maximum threads Needed in one pool Instance.
- Returns: 0 on success, negative on error.
- Functionality: Initializes internal structures & returns worker pool Handle
- Limitations: Must be called before other APIs. Not thread-safe.
- Concurrency: Call once at startup.

int wp_post_job(void (*job_func)(void *), void *arg);

- Args: Function pointer to job, Job argument pointer.
- Returns: Job ID or error code.
- Functionality: Adds job to job queue, allocates thread if available or wait till thread get's free
- Limitations: If threads in pool is at max, job waits in queue.
- Concurrency: Safe to call concurrently.

int wp_collect_job(int job_id);

- Args: Job ID.
- Returns: Status of job completion.
- Functionality: Blocks until job completes, retrieves result.
- Limitations: Caller must know job ID. Collecting same job by multiple threads will lead to failure of other.
- Concurrency: Multiple threads can collect different jobs.

void wp_deinit(void);

- Args: None.
- Functionality: Cleans up threads, frees handles, destroys semaphores.
- Limitations: Must be called once at shutdown.
- Concurrency: Not safe to call while jobs are running.

4. Test suite validation

The **test suite** is designed to rigorously validate the correctness, stability, and concurrency behavior of the Worker Pool library. It ensures that the pool behaves predictably under different workloads, thread counts, and error conditions.

Purpose

The test suite verifies that:

- Thread creation and reuse follow the configured limits.
- Job submission and completion are concurrency-safe.
- Multiple pools can coexist independently in a single process.

5. Known Limitations

This section lists current, known limitations of the worker-pool implementation and practical consequences for integrators. Each item explains the issue, the likely impact, and a short mitigation or workaround.

1. No per-thread monitor for abnormal termination

- **Issue:** There is no dedicated monitor that watches worker threads for abnormal termination (e.g., `pthread_exit()` called inside a job, or a job that crashes with a signal).
- **Impact:** If a worker thread terminates unexpectedly while executing a job, the corresponding `WP_collectJob()` call for that job can **hang** waiting for a completion that never arrives. The process may also print job-handled error messages (segfault traces) while the pool's bookkeeping remains inconsistent.

2. Threads run at system default priority; per-thread priority changes are persistent

- **Issue:** All threads are created with the system default scheduling policy and priority. A job can call "`pthread_setschedparam()`" to raise its thread priority, but the library does not automatically restore the original priority after the job finishes.
- **Impact:** If a job raises its thread priority and does not reset it, that thread will continue to run at the elevated priority for subsequent jobs, which can cause unfair scheduling and starvation of other tasks. Also, changing scheduling policy/priority often requires elevated privileges on many systems.

- **Mitigation:** If you need per-job priority changes, implement a wrapper that records the thread's original scheduling parameters, applies the requested change, and restores the original parameters before returning.

3. No built-in resource limits for queued jobs or memory usage

- **Issue:** The queue can grow unbounded if producers submit jobs faster than workers can consume them. There is no built-in backpressure or queue size limit.
- **Impact:** Unbounded queues can exhaust memory under heavy load.